

Remerciements

Je tiens à exprimer ma sincère gratitude à mon encadrant académique [*Nom*], pour ses précieux conseils et sa disponibilité tout au long de ce projet.

Je remercie également l'équipe d'ASM Technology pour m'avoir accueilli et fourni un environnement de travail stimulant.

Enfin, je remercie le corps enseignant de l'ISIMS pour la qualité de la formation dispensée durant ces années d'études.

Table des matières

Introduction générale	2
1 Étude préalable	4
1 Introduction	4
2 Présentation de l'organisme d'accueil	4
3 Cadre général du projet	4
4 Contexte et problématique	5
4.1 Études des applications existantes sur le marché	5
4.2 Critique des solutions existantes	6
4.3 Problématique	6
5 Solution proposée	7
6 Méthodologie de travail adoptée	7
6.1 Méthodologie Scrum	7
6.2 Backlog produit	9
6.3 Sprint planning	11
6.4 Outil de planification du travail	12
7 Conclusion	13
2 Spécification des besoins et analyse	14
1 Introduction	14
2 Étude fonctionnelle	14
2.1 Identification des acteurs	14
2.2 Identification des besoins fonctionnels	15
2.3 Identification des besoins non fonctionnels	16
2.4 Diagrammes de cas d'utilisation	17
2.5 Diagrammes de séquence système	26
3 Conclusion	27

Table des figures

1.1	Logo ASM	4
1.2	Logo Bringgg	5
1.3	Logo Onfleet	5
1.4	Les valeurs Scrum	8
1.5	Tableau Kanban GitLab — projet ASM Track	13
2.1	Diagramme de cas d'utilisation global — ASM Track	18
2.2	Diagramme raffiné — Création de tournée	19
2.3	Diagramme raffiné — Gestion et suivi des tournées	19
2.4	Diagramme raffiné — Exécution et confirmation de livraison	20
2.5	Diagramme raffiné — Création de tournée	21
2.6	Diagramme raffiné — Exécution et confirmation de livraison (Chauffeur)	24
2.7	Diagramme de séquence — Authentification administrateur	26

Liste des tableaux

1.1	Product Backlog — ASM Track	9
1.2	Réunion de planification des sprints	12
2.1	Description textuelle CU : Assigner une livraison	21
2.2	Description textuelle CU : Créer une tournée	22
2.3	Description textuelle CU : Valider une tournée	23
2.4	Description textuelle CU : Confirmer une livraison avec POD	24

Introduction générale

La logistique est aujourd'hui présente dans tous les secteurs économiques et constitue un maillon stratégique du bon fonctionnement des entreprises. La présence de systèmes informatisés de suivi des livraisons est devenue un critère essentiel pour toute organisation souhaitant rester compétitive. Pour cette raison, les entreprises sont prêtes à investir dans des solutions logicielles capables de centraliser et d'automatiser la gestion de leurs flux opérationnels. En effet, compte tenu de la croissance des volumes de livraisons et de la multiplicité des intervenants, la tâche de gérer efficacement les tournées, les chauffeurs et les synchronisations avec les systèmes ERP s'avère de plus en plus complexe. Il devient ainsi primordial que chaque opérateur logistique se dote d'une plateforme de gestion intégrée, flexible et adaptée à la diversité de ses environnements techniques.

Notre contribution consiste à réaliser un projet de fin d'études en collaboration avec l'entreprise **ASM**, basée à Sfax. L'objectif de ce projet est de concevoir et de développer **ASM Track**, une plateforme SaaS multi-tenant de gestion et de suivi des livraisons en temps réel. Cette solution permettra de planifier les tournées de livraison, de suivre en temps réel la position et l'état des colis, de synchroniser automatiquement les livraisons avec les systèmes ERP de l'entreprise cliente, et de centraliser les preuves de livraison. Cela permettra d'éliminer les saisies manuelles, de réduire les erreurs opérationnelles et d'offrir une meilleure visibilité à l'ensemble des parties prenantes.

Le présent rapport détaille l'avancement du projet qui s'étend sur huit chapitres, comme suit :

Chapitre 1 : Dans ce chapitre, nous introduisons l'entreprise d'accueil, présentons le cadre général du projet, analysons les solutions existantes sur le marché et exposons la méthodologie de travail adoptée.

Chapitre 2 : Ce chapitre présente les spécifications des besoins fonctionnels et non fonctionnels, qui constituent la phase d'analyse du projet. Nous y établissons les acteurs du système et les cas d'utilisation.

Chapitre 3 : Dans ce chapitre, nous décrivons l'architecture microservices retenue et justifions les choix technologiques adoptés pour le backend, le frontend et le mobile.

Chapitre 4 : Ce chapitre décrit le développement du premier sprint : le serveur d'authentification OAuth2 personnalisé et le cycle de vie complet des livraisons.

Chapitre 5 : Dans ce chapitre, nous décrivons le développement du deuxième sprint :

l'intégration ERP agnostique via le pattern Ports & Adapters et la résilience par le pattern Outbox transactionnel.

Chapitre 6 : Ce chapitre présente le troisième sprint : le constructeur de tournées, l'intégration du moteur de routage OSRM, le suivi temps réel via WebSocket et la surveillance des SLA.

Chapitre 7 : Dans ce chapitre, nous décrivons le quatrième sprint : l'application mobile Flutter pour les chauffeurs, avec mode hors-ligne, capture de preuves de livraison et transfert de tournée par QR code.

Chapitre 8 : Ce chapitre couvre le cinquième sprint : la mise en place de l'architecture SaaS multi-tenant, le panneau super-administrateur et le déploiement de la stack complète via Docker Compose.

Enfin, nous clôturons notre rapport par une conclusion générale exposant une synthèse du travail réalisé et ouvrant le champ à d'autres perspectives d'évolution pour ASM Track.

Chapitre 1

Étude préalable

1 Introduction

Ce chapitre constitue le socle du projet ASM Track. Nous y présentons l'organisme d'accueil, définissons le cadre général du projet, analysons les solutions existantes sur le marché, exposons la problématique identifiée, décrivons la solution proposée, puis détaillons la méthodologie de travail adoptée tout au long du développement.

2 Présentation de l'organisme d'accueil

Étant une Entreprise de Services du Numérique (ESN), **All Soft Multimédia (ASM)** est une structure souple et réactive. Elle propose des solutions sur mesure dans le domaine des nouvelles technologies et des systèmes d'information.



FIGURE 1.1 – Logo ASM

Créée en **2007**, ASM est une société spécialisée dans l'ingénierie logicielle, la création de sites web, le développement d'applications mobiles ainsi que les systèmes de points de vente. Elle est présente en Tunisie à **Sfax**, **Sousse** et **Tunis**, en Algérie à **Alger**, et au Sénégal à **Dakar**.

3 Cadre général du projet

Ce projet de fin d'études, réalisé au sein d'ASM, s'inscrit dans le cadre de l'obtention du **Diplôme National d'Ingénieur en Informatique** à l'Institut Supérieur d'Informatique et de Multimédia de Sfax (ISIMS).

L'objectif est de concevoir et de développer **ASM Track**, une plateforme SaaS multi-tenant de gestion et de suivi des livraisons en temps réel. Le projet couvre l'intégralité

de la chaîne logistique du dernier kilomètre, depuis l'importation des commandes depuis un ERP jusqu'à la livraison finale avec capture de preuve, en passant par la planification des tournées et le suivi en temps réel.

4 Contexte et problématique

Pour illustrer notre propos, nous menons dans ce qui suit une étude contextuelle partant de l'analyse des solutions de gestion des livraisons présentes sur le marché, allant vers leur critique, pour aboutir à la description de notre problématique.

4.1 Études des applications existantes sur le marché

Avant d'entamer notre projet, il est essentiel de réaliser une veille externe. C'est pourquoi il est important d'examiner les applications déjà présentes sur le marché afin de mieux comprendre les aspects relatifs à notre contexte et d'identifier les manques auxquels notre solution doit répondre.

Bringgg

Bringgg est une plateforme SaaS de gestion des livraisons du dernier kilomètre, fondée en Israël et opérant à l'échelle internationale. Elle propose une suite complète d'outils pour les opérateurs logistiques : planification des tournées, affectation des chauffeurs, suivi GPS en temps réel et notifications automatiques aux destinataires. La plateforme se distingue par son interface intuitive et ses capacités d'intégration avec des systèmes de e-commerce tels que Shopify et WooCommerce, ce qui lui permet de répondre aux besoins des entreprises de distribution à grande échelle.



FIGURE 1.2 – Logo Bringgg

Onfleet

Onfleet est une solution américaine de gestion des livraisons destinée aux entreprises de taille moyenne et grande. Elle offre des fonctionnalités avancées de dispatch automatique, de suivi en temps réel, d'analyse des performances et de communication entre les dispatchers et les chauffeurs via une application mobile dédiée. Onfleet est reconnue pour la qualité de son API REST, qui facilite son intégration avec des systèmes tiers, et pour ses tableaux de bord analytiques permettant de suivre les indicateurs clés de performance des équipes de livraison.



FIGURE 1.3 – Logo Onfleet

4.2 Critique des solutions existantes

Dans cette section, nous effectuons une critique approfondie des solutions étudiées. Nous examinons leurs points forts ainsi que leurs limites, afin d'identifier les opportunités auxquelles notre solution personnalisée doit répondre.

Bringgg

Bringgg propose une plateforme moderne et extensible avec des fonctionnalités avancées de suivi temps réel. Toutefois, l'intégration avec les ERP utilisés par les entreprises tunisiennes, tels qu'Odoo ou Dux, nécessite généralement des développements spécifiques ou l'utilisation de solutions tierces telles que Zapier ou Make, ce qui augmente la complexité et le coût d'intégration. De plus, Bringgg cible principalement les grandes entreprises avec un modèle tarifaire sur devis personnalisé, sans grille de prix publique, ce qui le rend peu accessible pour les PME et les entreprises en croissance.

Onfleet

Onfleet offre une interface ergonomique et de bonnes capacités de suivi opérationnel. Cependant, l'intégration avec les ERP n'est pas native dans le contexte des solutions utilisées en Tunisie et repose principalement sur des APIs ou des outils d'automatisation tiers, ce qui nécessite des développements supplémentaires et une maintenance accrue. Par ailleurs, son modèle tarifaire basé sur le volume de livraisons peut devenir coûteux pour les entreprises à forte activité.

Il est important de souligner que la solution que nous souhaitons développer doit pouvoir se connecter à n'importe quel ERP — qu'il s'agisse d'Odoo, de Dux ou d'un autre système à l'avenir — sans avoir à tout réécrire à chaque changement. Elle doit également permettre de gérer plusieurs entreprises clientes depuis une seule plateforme, avec des données totalement séparées pour chacune.

4.3 Problématique

Face aux limites des solutions existantes, les entreprises expriment des besoins clairs auxquels aucune solution du marché ne répond pleinement :

- Savoir en temps réel où se trouvent leurs livraisons et être alertées immédiatement en cas de retard ou de dépassement de délai.
- Connecter automatiquement leur système de gestion (ERP) à la plateforme de livraison, quel que soit l'ERP utilisé, sans ressaisie manuelle des commandes.
- Donner à leurs chauffeurs une application mobile simple, utilisable même sans connexion internet, pour confirmer les livraisons et capturer des preuves.

- Disposer d’une solution qui s’adapte à leur structure, qu’ils aient une seule agence ou plusieurs, sans multiplier les abonnements ni les installations.

5 Solution proposée

Pour répondre à cette problématique, nous proposons **ASM Track**, une plateforme SaaS multi-tenant de gestion des livraisons articulée autour de cinq axes principaux :

- **Architecture microservices** : six services indépendants déployés via Docker Compose, communiquant par REST et WebSocket.
- **Intégration ERP flexible** : une architecture permettant de se connecter à n’importe quel ERP (Odoo, Dux, ou tout futur système) sans modifier le cœur de la plateforme.
- **Fiabilité** : un mécanisme garantissant qu’aucune synchronisation avec l’ERP n’est perdue, même en cas de panne réseau ou de redémarrage du serveur.
- **Constructeur de tournées** : un éditeur graphique permettant au dispatcher de planifier les tournées par glisser-déposer, avec calcul automatique des itinéraires optimaux.
- **Suivi en temps réel** : un tableau de bord pour les dispatchers affichant l’état de chaque livraison en direct, avec alertes automatiques en cas de dépassement de délai.
- **Application mobile chauffeur** : une application dédiée aux chauffeurs, fonctionnelle même sans connexion internet, permettant de confirmer les livraisons, capturer des preuves et transférer une tournée à un autre chauffeur via QR code.
- **SaaS multi-tenant** : plusieurs entreprises clientes utilisent la même plateforme avec des données totalement séparées, et ASM dispose d’un panneau d’administration centralisé pour gérer l’ensemble des clients.

6 Méthodologie de travail adoptée

6.1 Méthodologie Scrum

Face à la diversité des fonctionnalités à développer et à l’évolution continue des besoins au cours du projet, il était indispensable de choisir une méthodologie capable de s’adapter rapidement et de livrer de la valeur de façon régulière. Notre choix s’est porté sur **Scrum**, un cadre agile de gestion de projet reconnu pour sa capacité à structurer le développement logiciel en cycles courts et itératifs, tout en maintenant une vision claire des priorités.

Définition de Scrum

Scrum est un cadre de travail empirique fondé sur trois piliers : la transparence, l'inspection et l'adaptation. Contrairement aux approches traditionnelles en cascade où tout est planifié à l'avance, Scrum découpe le développement en cycles courts appelés sprints, à l'issue desquels un incrément fonctionnel est livré. Pour ASM Track, ce mode de fonctionnement nous a permis de valider chaque brique technique — authentification, ERP, mobile — avant de passer à la suivante, sans risquer de construire sur des bases instables.

Le cadre Scrum repose sur trois rôles complémentaires :

- Le **Product Owner** est responsable de définir et de prioriser les fonctionnalités du produit en fonction de leur valeur métier. Il maintient le Product Backlog à jour et s'assure que l'équipe travaille sur les éléments les plus importants à chaque sprint.
- Le **Scrum Master** veille au respect du cadre Scrum et à la bonne application de ses pratiques. Il identifie et lève les obstacles qui freinent la progression de l'équipe, sans jouer le rôle d'un chef de projet traditionnel.
- L'**équipe de développement** est responsable de la conception, de l'implémentation et des tests des fonctionnalités sélectionnées à chaque sprint. Elle s'auto-organise pour atteindre l'objectif fixé.

Scrum s'appuie également sur cinq valeurs — l'engagement, le courage, le focus, l'ouverture et le respect — qui ont guidé nos décisions tout au long du projet, notamment lors des arbitrages techniques entre différentes approches d'implémentation.



FIGURE 1.4 – Les valeurs Scrum

Événements Scrum

Scrum définit cinq événements récurrents qui structurent le travail à l'intérieur de chaque sprint :

- **Le Sprint** est la période de travail pendant laquelle l'équipe s'engage à produire un incrément fonctionnel et potentiellement livrable. Sa durée est fixe et ne dépasse pas un mois. Dès la clôture d'un sprint, le suivant commence immédiatement, assurant ainsi un rythme de développement continu.
- **Le Sprint Planning** ouvre chaque sprint. L'équipe y sélectionne les éléments du Product Backlog à réaliser, estime leur complexité et définit un objectif de sprint clair et partagé par tous les membres.
- **Le Daily Scrum** est une réunion quotidienne de quinze minutes destinée à synchroniser l'équipe, identifier les obstacles éventuels et ajuster le plan de travail de la journée en fonction de l'avancement réel.
- **La Sprint Review** se tient à la fin de chaque sprint. L'incrément développé y est présenté aux parties prenantes, dont les retours permettent d'affiner le Product Backlog et d'orienter les priorités des sprints suivants.
- **La Sprint Retrospective** clôture chaque sprint sur le plan organisationnel. L'équipe y analyse son fonctionnement, identifie les points d'amélioration et définit des actions concrètes pour progresser dans le sprint suivant.

6.2 Backlog produit

Le Product Backlog regroupe l'ensemble des fonctionnalités identifiées lors de la phase d'analyse. Chaque entrée est décrite par son titre, sa description, ses critères d'acceptation, sa priorité et son estimation en jours :

TABLE 1.1 – Product Backlog — ASM Track

N°	Titre	Description	Critères d'acceptation	Priorité	Est. (J)
1	Serveur OAuth2	Mise en place d'un serveur d'authentification OAuth2 avec JWT signé RSA	Les tokens sont émis via /oauth2/token, signés RSA, et validés par l'API Gateway via JWKS	Haute	3
2	Connexion admin	L'admin se connecte avec son email et mot de passe	Token d'accès JWT émis, rôle et identifiant entreprise embarqués	Haute	3
3	Connexion chauffeur	Le chauffeur se connecte avec son numéro de téléphone	Token JWT émis, accès à l'espace mobile autorisé	Haute	3

N°	Titre	Description	Critères d'acceptation	Priorité	Est. (J)
4	Cycle de vie livraison	Gestion complète du cycle d'une livraison de la création à la clôture	9 états gérés, de UNSCHEDULED jusqu'à DELIVERED, FAILED ou CANCELLED, historique enregistré	Haute	3
5	Assignation chauffeur	L'admin assigne une livraison à un chauffeur disponible	Statut passe à SCHEDULED, notification push FCM envoyée au chauffeur	Haute	3
6	Import commandes	Les commandes sont importées depuis Odoo ou Dux	Les livraisons apparaissent en statut UNSCHEDULED dans le tableau de bord	Haute	4
7	Synchronisation ERP	Les événements de livraison sont synchronisés vers l'ERP de manière fiable	L'ERP est mis à jour automatiquement, idempotence garantie en cas de retry	Haute	4
8	Adaptateur multi-ERP	Basculer entre Odoo et Dux sans modifier le code métier	Le changement d'ERP se fait par configuration, sans redéploiement	Haute	4
9	Constructeur de tournées	Le dispatcher planifie les tournées par glisser-déposer	L'itinéraire est calculé via OSRM (carte Tunisie), les arrêts sont ordonnés	Haute	5
10	Suivi temps réel	Le dispatcher voit l'état des livraisons et la position des chauffeurs en direct	Les mises à jour sont poussées via WebSocket STOMP/RabbitMQ sans rechargement	Haute	5

N°	Titre	Description	Critères d'acceptation	Priorité	Est. (J)
11	Monitoring SLA	Détection automatique des livraisons en dépassement de délai	Alerte envoyée au dispatcher en moins de 60 secondes	Moyenne	3
12	Mode hors-ligne	Le chauffeur continue à enregistrer des livraisons sans connexion internet	Les actions sont stockées localement (Hive) et synchronisées au retour du réseau	Haute	4
13	Preuve de livraison	Le chauffeur capture une photo et une signature à la livraison	Le fichier est stocké sur MinIO et visible par l'admin	Haute	4
14	Transfert QR	Le chauffeur transfère une tournée à un collègue via QR code	Le chauffeur cible reçoit les stops instantanément	Moyenne	2
15	Rapport PDF	L'admin génère un rapport PDF de tournée avec les KPIs	Le PDF contient 9 indicateurs de performance et est téléchargeable	Moyenne	3
16	Multi-tenant SaaS	Plusieurs entreprises utilisent la plateforme avec données isolées	Chaque entreprise voit uniquement ses données via filtre Hibernate	Haute	4
17	Déploiement Docker	L'ensemble de la stack est déployé via Docker Compose	Tous les services démarrent avec healthchecks et réseau isolé	Haute	4

6.3 Sprint planning

Sur la base du Product Backlog établi, nous avons découpé le projet en cinq sprints successifs, chacun centré sur un axe fonctionnel précis. Ce découpage nous a permis de valider progressivement les choix techniques et de livrer des incréments testables à

chaque étape :

TABLE 1.2 – Réunion de planification des sprints

Sprint	Description	Date début	Date fin
Sprint 1	Authentification et cycle de vie des livraisons	28/02/2025	21/03/2025
Sprint 2	Intégration ERP agnostique (Odoo / Dux)	22/03/2025	11/04/2025
Sprint 3	Constructeur de tournées et suivi temps réel	12/04/2025	02/05/2025
Sprint 4	Application mobile chauffeur	03/05/2025	23/05/2025
Sprint 5	Reporting, multi-tenancy et déploiement	24/05/2025	30/06/2025

6.4 Outil de planification du travail

Le versioning du code et le suivi des tâches ont été assurés via **GitLab**. Au-delà du contrôle de version, GitLab nous a offert un environnement de gestion de projet complet grâce à plusieurs fonctionnalités intégrées :

Suivi intégré des tâches : GitLab permet de suivre les tâches via des issues (tickets). Chaque issue peut être associée à des étiquettes, des assignés, des échéances et des commentaires, offrant ainsi une vue claire de l'état d'avancement du projet.

Relation entre les commits et les merge requests : GitLab permet de lier directement les commits et les merge requests aux issues. Chaque changement de code est ainsi traçable jusqu'à une tâche spécifique, ce qui améliore la transparence et la responsabilité.

Tableaux Kanban : Les tableaux de projet de GitLab permettent de visualiser le flux de travail sous forme de tableaux Kanban. Nous avons configuré notre tableau avec les colonnes suivantes : *To do*, *In progress*, *In review* et *Done*. Cela aide à identifier les goulots d'étranglement et à améliorer le flux de travail.

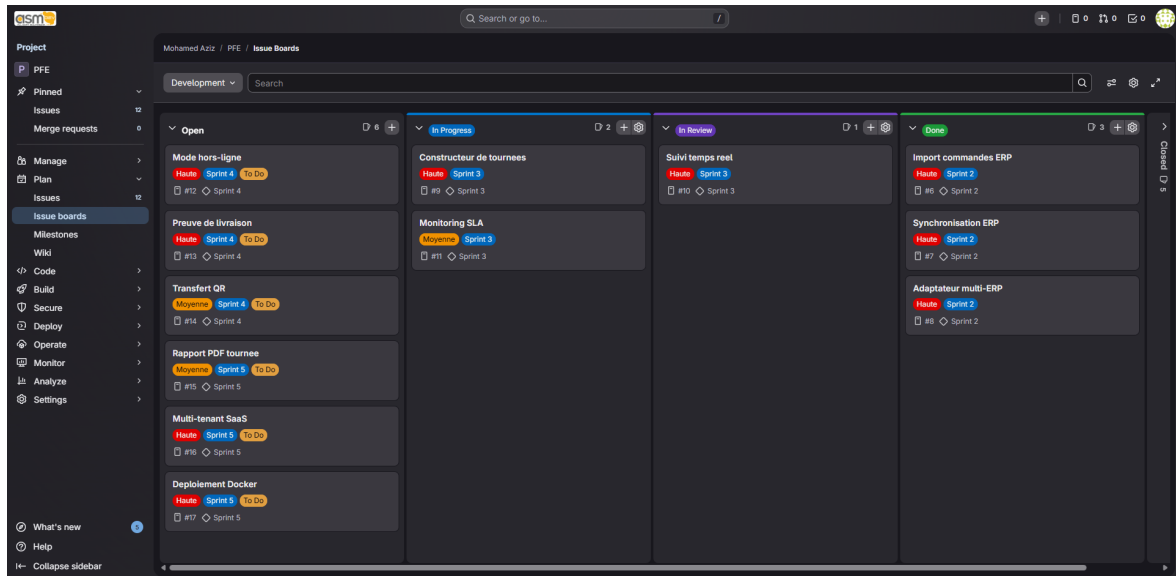


FIGURE 1.5 – Tableau Kanban GitLab — projet ASM Track

En résumé, GitLab offre une plateforme unifiée pour la gestion des tâches, le suivi des commits et les revues de code, rendant le processus de développement plus transparent et collaboratif.

7 Conclusion

Ce chapitre a constitué le point de départ de notre projet. Nous avons commencé par présenter l'organisme d'accueil ASM, puis nous avons mené une étude des solutions existantes sur le marché — Bringgg et Onfleet — dont l'analyse critique nous a permis d'identifier les limites auxquelles aucune d'elles ne répond pleinement. Sur cette base, nous avons formulé notre problématique et décrit la solution proposée : ASM Track, une plateforme SaaS multi-tenant de gestion des livraisons. Enfin, nous avons présenté la méthodologie Scrum adoptée, avec son backlog produit de dix-sept user stories réparties sur cinq sprints et gérées via GitLab. Le chapitre suivant présente l'analyse et la spécification détaillée des besoins fonctionnels et non fonctionnels de la plateforme.

Chapitre 2

Spécification des besoins et analyse

1 Introduction

Après avoir posé le cadre général du projet et défini la méthodologie de travail adoptée, nous entamons dans ce chapitre la phase d'analyse fonctionnelle d'ASM Track. Cette étape consiste à identifier les acteurs du système, à recenser leurs besoins fonctionnels et non fonctionnels, puis à les formaliser sous forme de diagrammes de cas d'utilisation et de diagrammes de séquence. Cette analyse constitue le fondement sur lequel reposent tous les choix de conception et d'implémentation qui seront détaillés dans les chapitres suivants.

2 Étude fonctionnelle

L'étude fonctionnelle est une phase indispensable pour identifier les tâches que le système doit accomplir et les interactions entre ses différents utilisateurs. Elle se traduit par la liste des acteurs, leurs besoins et leurs interactions, formalisés sous forme de diagrammes UML.

2.1 Identification des acteurs

Un acteur représente toute entité externe qui interagit directement avec le système. Nous avons identifié quatre acteurs pour ASM Track :

- **Le Super Admin** : représente l'équipe d'ASM. Il dispose d'un accès global à la plateforme, sans restriction par entreprise. Il est responsable de la gestion des entreprises clientes, des chauffeurs et des utilisateurs administrateurs.
- **L'Admin / Dispatcher** : représente un utilisateur appartenant à une entreprise cliente. Il gère les livraisons, planifie les tournées, suit l'avancement en temps réel et génère les rapports. Ses données sont strictement isolées de celles des autres entreprises.
- **Le Chauffeur** : représente un livreur sur le terrain. Il interagit exclusivement via l'application mobile Flutter. Il consulte ses livraisons assignées, confirme les étapes

du cycle de livraison et capture les preuves de livraison, même sans connexion internet.

- **Le Système ERP** : représente le système de gestion de l'entreprise cliente (Odoo ou Dux). C'est un acteur externe qui envoie des commandes vers ASM Track et reçoit en retour les mises à jour de statut des livraisons.

2.2 Identification des besoins fonctionnels

En tant que Super Admin, je peux :

- Créer, modifier, activer et désactiver des entreprises clientes.
- Créer et gérer les profils des chauffeurs (nom, téléphone, statut).
- Créer et gérer les comptes administrateurs des entreprises clientes.
- Réinitialiser les mots de passe des administrateurs.
- Consulter les statistiques globales de la plateforme.
- Accéder aux données de toutes les entreprises sans restriction de tenant.

En tant qu'Admin / Dispatcher, je peux :

- M'authentifier avec mon email et mon mot de passe.
- Consulter et rechercher les commandes non planifiées par statut, date ou source.
- Replanifier une commande en la retirant d'une tournée et en la réassignant à une autre.
- Annuler une commande avant qu'elle ne soit prise en charge par un chauffeur.
- Accéder au constructeur de tournées et visualiser les commandes non planifiées disponibles.
- Créer une tournée et y ajouter des commandes par glisser-déposer.
- Optimiser l'itinéraire de la tournée automatiquement via OSRM.
- Assigner une tournée à un chauffeur disponible.
- Suivre en temps réel l'avancement des tournées et la position des chauffeurs.
- Recevoir des alertes automatiques en cas de dépassement de délai SLA.
- Consulter les preuves de livraison (photos et signatures) capturées par les chauffeurs lors de l'exécution de la tournée.
- Générer un rapport PDF d'une tournée clôturée avec ses indicateurs de performance.
- Configurer les seuils SLA de l'entreprise (délai d'attente, délai d'assignation, délai de ramassage) et les modifier dynamiquement en temps réel.

En tant que Chauffeur, je peux :

- M'authentifier avec mon numéro de téléphone et mon mot de passe.
- Consulter ma tournée du jour avec la liste ordonnée des arrêts à effectuer.
- Démarrer ma tournée et naviguer vers chaque arrêt grâce à l'itinéraire calculé.
- Confirmer le ramassage des colis d'un arrêt (statut `PICKED_UP`).
- Démarrer le transit vers le destinataire (statut `IN_TRANSIT`).
- Confirmer la livraison complète d'un arrêt avec capture de photo et de signature (POD).
- Confirmer une livraison partielle lorsqu'une partie seulement des articles a été remise au destinataire (statut `PARTIALLY_DELIVERED`).
- Marquer un arrêt comme échoué avec indication de motif.
- Travailler en mode hors-ligne : toutes mes actions sur la tournée sont sauvegardées localement et synchronisées automatiquement dès le retour du réseau.
- Transférer ma tournée à un autre chauffeur via QR code.
- Mettre à jour ma position GPS pour permettre le suivi en temps réel par le dispatcher.

En tant que Système ERP, je peux :

- Envoyer des commandes confirmées vers ASM Track pour les transformer en livraisons.
- Recevoir automatiquement les mises à jour de statut des livraisons (livrée, partiellement livrée, échouée, annulée) ainsi que les ajustements de stock correspondants, sans intervention manuelle.

2.3 Identification des besoins non fonctionnels

Les besoins non fonctionnels définissent les critères de qualité que le système doit respecter indépendamment des fonctionnalités :

- **La sécurité** : toutes les communications sont protégées par HTTPS. L'authentification repose sur un serveur OAuth2 personnalisé émettant des tokens JWT signés avec une clé RSA. L'API Gateway valide chaque token avant de router la requête. L'isolation des données entre entreprises est assurée par des filtres Hibernate appliqués automatiquement.
- **La performance** : les mises à jour temps réel doivent être poussées en moins de 2 secondes. Les pages du tableau de bord doivent se charger en moins de 3 secondes dans des conditions réseau normales.

- **La disponibilité** : la plateforme doit être disponible en continu. Chaque micro-service dispose d'un healthcheck Docker et redémarre automatiquement en cas de défaillance.
- **La fiabilité** : aucun événement métier ne doit être perdu. La synchronisation ERP est garantie même en cas de panne réseau ou de redémarrage d'un service.
- **L'idempotence** : les opérations critiques telles que la synchronisation ERP et la soumission de preuves de livraison (POD) peuvent être rejouées sans effet de bord. Une même requête exécutée plusieurs fois produit toujours le même résultat, évitant ainsi les doublons en cas de retry automatique.
- **La scalabilité** : l'architecture microservices permet de dimensionner indépendamment chaque service selon la charge. Le modèle multi-tenant permet d'intégrer de nouveaux clients sans déploiement supplémentaire.
- **La maintenabilité** : le code métier est isolé des dépendances externes grâce à une architecture en couches où chaque système tiers (ERP, stockage, routage) est remplaçable sans modifier le reste de la plateforme.
- **L'accessibilité mobile** : l'application chauffeur doit fonctionner avec ou sans connexion internet. Les données critiques sont stockées localement et synchronisées automatiquement au retour du réseau.

2.4 Diagrammes de cas d'utilisation

Le diagramme de cas d'utilisation décrit le comportement du système du point de vue de l'utilisateur. Il exprime les fonctionnalités offertes par le système et les interactions entre les acteurs et ces fonctionnalités.

Diagramme de cas d'utilisation global

La figure 2.1 présente le diagramme de cas d'utilisation global d'ASM Track. Il illustre l'ensemble des interactions entre les quatre acteurs du système — Super Admin, Admin/Dispatcher, Chauffeur et Système ERP — et les fonctionnalités qu'ils peuvent réaliser. Les relations «*include*» indiquent les cas d'utilisation obligatoirement exécutés (notamment l'authentification), tandis que les relations «*extend*» représentent les fonctionnalités optionnelles déclenchées sous certaines conditions.

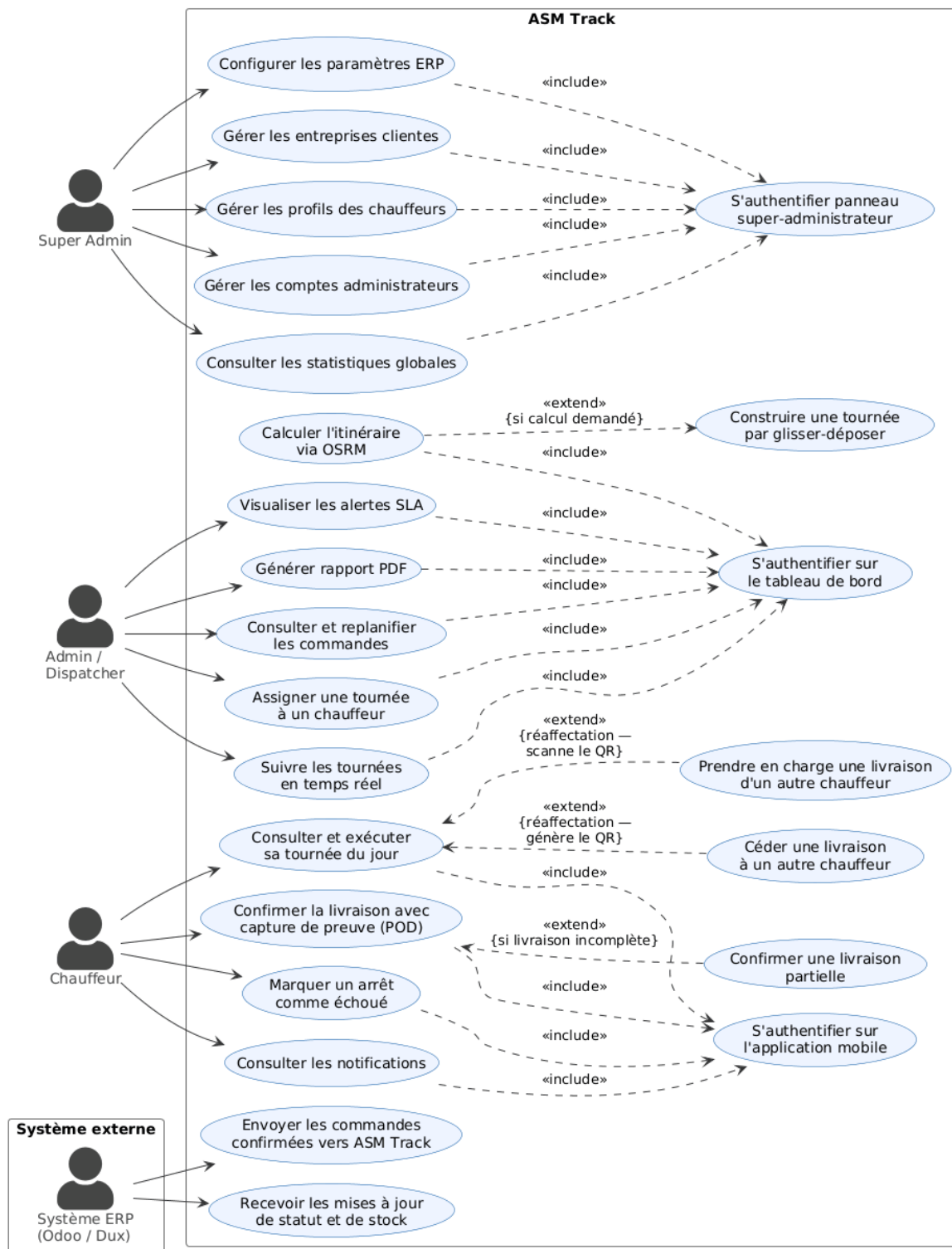


FIGURE 2.1 – Diagramme de cas d'utilisation global — ASM Track

Diagramme raffiné — Création de tournée

La figure suivante présente le diagramme raffiné de la création de tournée, illustrant les actions disponibles pour le dispatcher lors de la planification.

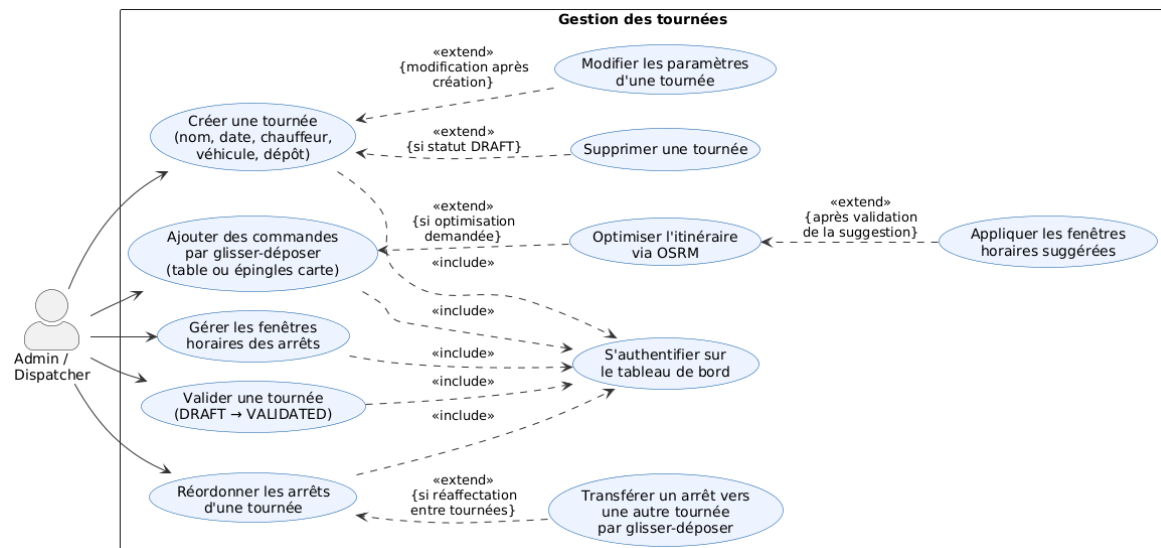


FIGURE 2.2 – Diagramme raffiné — Création de tournée

Diagramme raffiné — Gestion et suivi des tournées

La figure suivante présente le diagramme raffiné de la gestion et du suivi des tournées, couvrant les actions du dispatcher lors de l'exécution en temps réel.

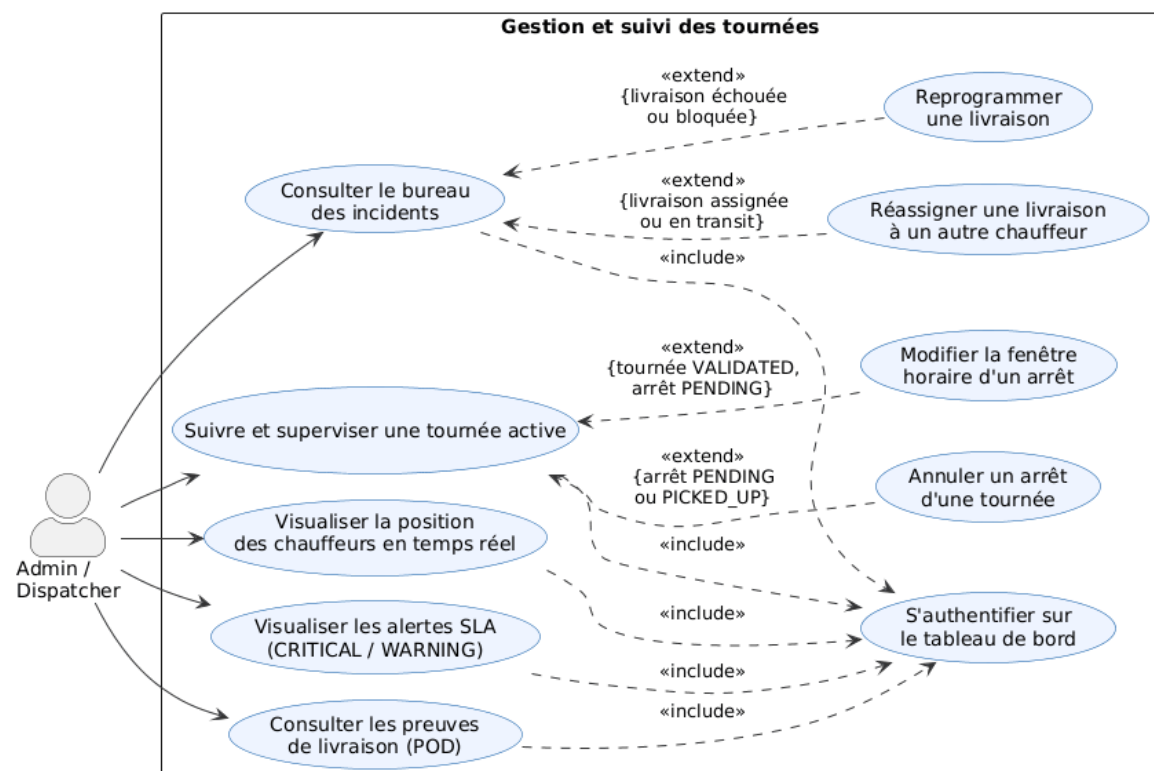


FIGURE 2.3 – Diagramme raffiné — Gestion et suivi des tournées

Diagramme raffiné — Exécution et confirmation de livraison

La figure suivante présente le diagramme raffiné du côté chauffeur, illustrant le cycle d'exécution d'une tournée et les actions de confirmation de livraison.

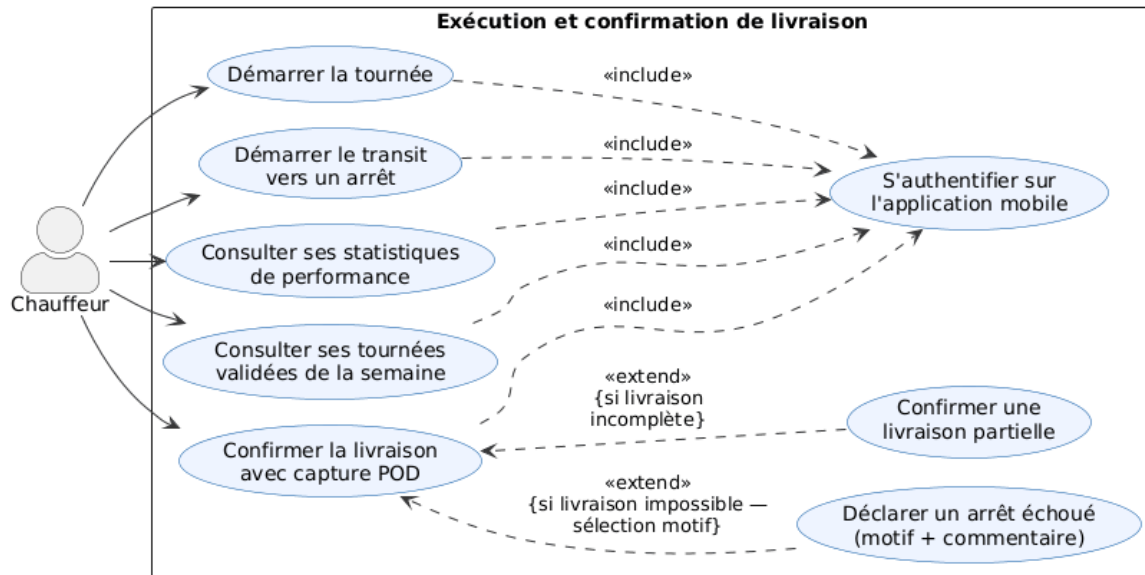


FIGURE 2.4 – Diagramme raffiné — Exécution et confirmation de livraison

Description textuelle CU : Assigner une livraison

CU : Assigner une livraison à un chauffeur	
Acteur	Admin / Dispatcher
Précondition	L'admin est authentifié ; la livraison est en statut UNSCHEDULED
Enchaînement principal :	
1. L'admin sélectionne une livraison dans le tableau de bord. 2. Le système affiche les chauffeurs disponibles. 3. L'admin choisit un chauffeur et confirme l'assignation. 4. Le système met à jour le statut à SCHEDULED et envoie une notification push FCM au chauffeur. 5. Le système enregistre l'événement dans l'historique des statuts.	
Postcondition	La livraison est assignée ; le chauffeur reçoit une notification sur son mobile
Enchaînement alternatif	Si aucun chauffeur n'est disponible, le système affiche un message d'avertissement et l'admin peut choisir de planifier l'assignation ultérieurement.

TABLE 2.1 – Description textuelle CU : Assigner une livraison

Diagramme raffiné — Planification des tournées

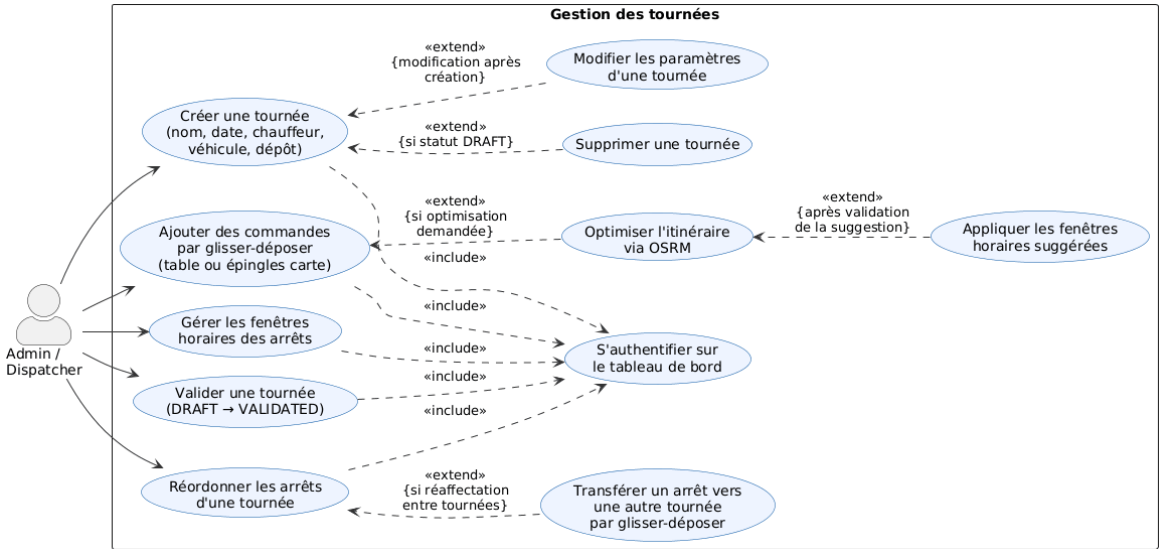


FIGURE 2.5 – Diagramme raffiné — Création de tournée

Description textuelle CU : Créer une tournée

TABLE 2.2 – Description textuelle CU : Créer une tournée

CU : Créer une tournée	
Acteur	Admin / Dispatcher
Précondition	L'admin est authentifié sur le tableau de bord ; des commandes en statut UNSCHEDULED existent ; au moins un chauffeur, un véhicule et un dépôt sont disponibles.
<i>Enchaînement principal :</i>	
1. L'admin ouvre le constructeur de tournées et clique sur "<i>Nouvelle tournée</i>". 2. Le système affiche un formulaire de création. L'admin saisit le nom de la tournée, la date d'opération, sélectionne le chauffeur, le véhicule et le dépôt de départ. 3. Le système crée la tournée en statut DRAFT et affiche le tableau des commandes non planifiées. 4. L'admin glisse-dépose les commandes depuis le tableau ou depuis les épingles de la carte vers la tournée. 5. Le système ajoute chaque commande comme arrêt et met à jour le compteur de charge (poids total / capacité véhicule). 6. L'admin clique sur "<i>Optimiser</i>" pour calculer l'itinéraire optimal via le moteur OSRM. 7. Le système retourne une suggestion d'ordre des arrêts avec les distances, durées, fenêtres horaires optimisées et l'heure d'arrivée estimée (ETA) pour chaque arrêt. 8. L'admin consulte la suggestion et clique sur "<i>Appliquer</i>". 9. Le système enregistre l'ordre optimisé et les fenêtres horaires suggérées pour chaque arrêt.	
Postcondition	La tournée est créée en statut DRAFT avec les arrêts ordonnés, les fenêtres horaires calculées et les ETAs enregistrés.

Enchaînement alternatif	A1 — Véhicule en surcharge : Si le poids total des commandes dépasse la capacité du véhicule, le système affiche une alerte rouge. L'admin peut retirer des commandes ou changer de véhicule. A2 — OSRM indisponible : Si le moteur de routage ne répond pas, le système conserve l'ordre manuel des arrêts et affiche un avertissement. L'admin peut relancer l'optimisation ultérieurement.
--------------------------------	---

Description textuelle CU : Valider une tournée

TABLE 2.3 – Description textuelle CU : Valider une tournée

CU : Valider une tournée	
Acteur	Admin / Dispatcher
Précondition	L'admin est authentifié ; la tournée est en statut DRAFT ; tous les arrêts disposent de fenêtres horaires complètes ; aucune violation chronologique n'est détectée.
<i>Enchaînement principal :</i>	
1. L'admin ouvre la tournée dans le constructeur et clique sur <i>"Finaliser et valider"</i>. 2. Le système ouvre une fenêtre de confirmation affichant pour chaque arrêt : le numéro, le client, l'adresse, les horaires prévus et un indicateur vert ou rouge selon la validité des fenêtres. 3. Le système vérifie automatiquement que chaque arrêt commence après la fin du précédent et qu'aucun chevauchement n'existe. 4. Si toutes les vérifications sont satisfaites, le bouton <i>"Valider la tournée"</i> est activé et affiché en vert. 5. L'admin clique sur <i>"Valider la tournée"</i>. 6. Le système sauvegarde les fenêtres horaires finales et fait passer le statut de la tournée de DRAFT à VALIDATED. 7. Le système envoie une notification push FCM au chauffeur assigné pour l'informer que sa tournée est prête.	

Postcondition	La tournée est en statut VALIDATED , les fenêtres horaires sont verrouillées et le chauffeur a reçu une notification.
Enchaînement alternatif	A1 — Violations chronologiques détectées : Si un ou plusieurs arrêts présentent des chevauchements ou des incohérences d’horaires, le système les surligne en rouge et désactive le bouton de validation. L’admin doit corriger les fenêtres horaires concernées avant de pouvoir valider. A2 — Fenêtres horaires incomplètes : Si un arrêt ne possède pas de fenêtre horaire définie, le système bloque la validation et indique les arrêts manquants.

Diagramme raffiné — Exécution et confirmation de livraison

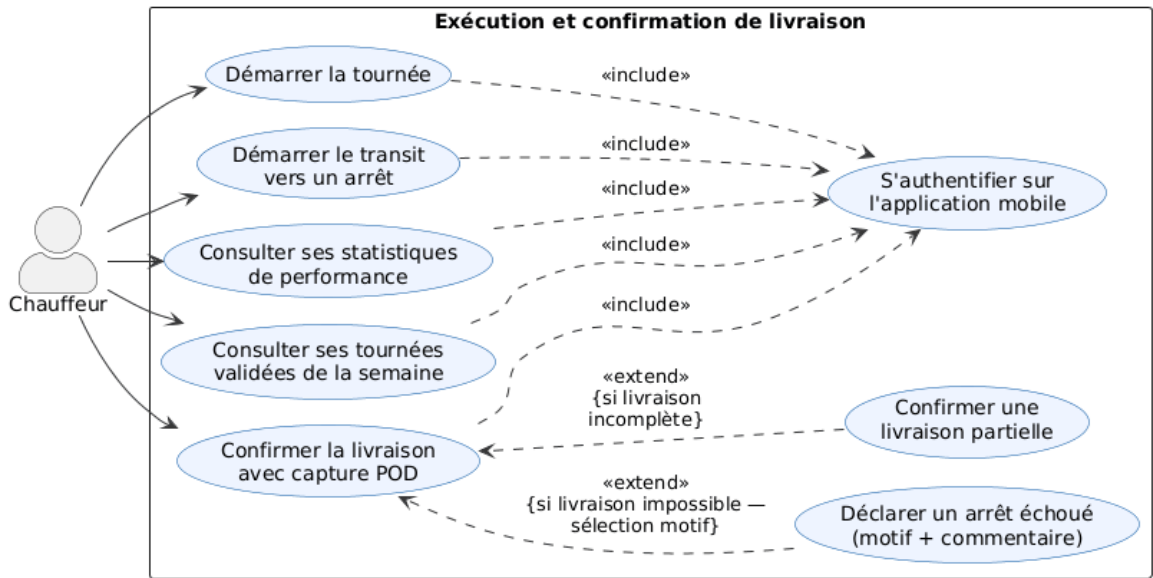


FIGURE 2.6 – Diagramme raffiné — Exécution et confirmation de livraison (Chauffeur)

Description textuelle CU : Confirmer une livraison avec capture de preuve (POD)

CU : Confirmer une livraison avec capture de preuve (POD)	
Acteur	Chauffeur

Précondition	Le chauffeur est authentifié sur l'application mobile ; la tournée est en statut IN_PROGRESS ; l'arrêt concerné est en statut IN_TRANSIT .
<i>Enchaînement principal :</i>	
1. Le chauffeur consulte sa tournée du jour et sélectionne l'arrêt en cours dans l'application mobile. 2. Le chauffeur démarre le transit vers l'arrêt. Le système met à jour le statut de l'arrêt à IN_TRANSIT. 3. À l'arrivée chez le destinataire, le chauffeur ouvre le formulaire de confirmation de livraison. 4. Le chauffeur imprime le bon de livraison (BL) et le fait signer par le destinataire. 5. Le chauffeur capture deux photos via l'application mobile : une photo du colis remis et une photo du bon de livraison signé. 6. L'application envoie les deux photos au serveur comme preuve de livraison (POD). 7. Le serveur stocke les photos sur MinIO et génère des URLs d'accès sécurisées. 8. Le système met à jour le statut de l'arrêt à DELIVERED et enregistre l'horodatage de livraison. 9. Le système publie un événement sur RabbitMQ. L'adaptateur ERP correspondant synchronise automatiquement la livraison dans le système de l'entreprise (Odoo ou Dux). 10. Le dispatcher voit instantanément le statut mis à jour sur son tableau de bord, sans avoir à recharger la page.	
Postcondition	L'arrêt est en statut DELIVERED , la preuve de livraison est stockée sur MinIO, l'ERP est synchronisé et le dispatcher est notifié en temps réel.

Enchaînement alternatif	A1 — Mode hors-ligne : Si le chauffeur n’a pas de connexion internet, l’application stocke la POD localement dans la file d’attente Hive. Dès que la connexion est rétablie, la synchronisation s’effectue automatiquement (3 tentatives maximum, TTL 24h). A2 — Livraison partielle : Si une partie seulement des articles a été remise, le chauffeur sélectionne les articles livrés et confirme une livraison partielle. Le statut passe à PARTIALLY_DELIVERED et l’ERP est notifié des quantités réellement livrées.
--------------------------------	---

2.5 Diagrammes de séquence système

Les diagrammes de séquence modélisent les interactions entre les composants du système lors de l’exécution d’un cas d’utilisation précis. Ils illustrent l’ordre chronologique des échanges de messages entre les acteurs et les services.

Diagramme de séquence — Authentification admin

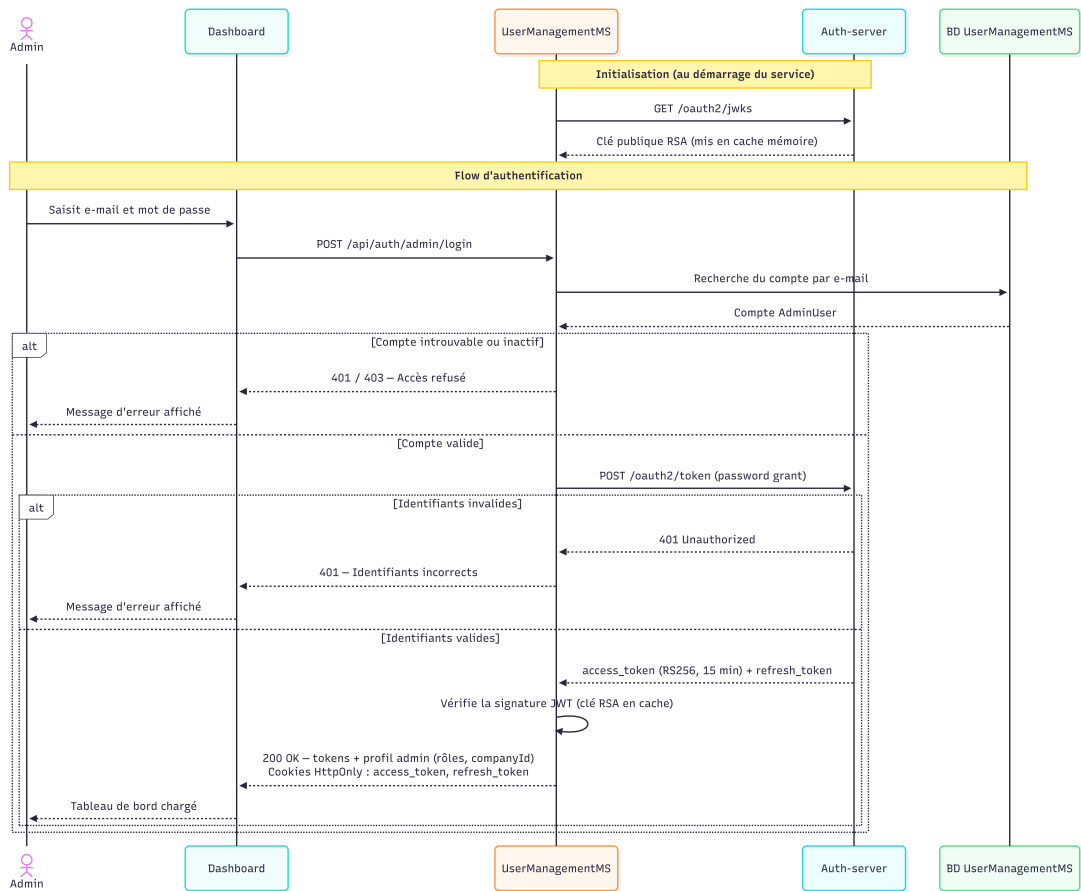


FIGURE 2.7 – Diagramme de séquence — Authentification administrateur

Lorsque l'administrateur soumet ses identifiants, le *UserManagementMS* vérifie d'abord l'existence et l'état actif du compte en base de données, puis délègue la validation du mot de passe au serveur d'autorisation via un *password grant* OAuth2. Au démarrage du service, la clé publique RSA est récupérée depuis l'endpoint JWKS et mise en cache mémoire, ce qui permet de vérifier la signature des tokens sans aller-retour réseau. En cas de succès, le token JWT (RS256, 15 min) et le profil de l'utilisateur sont retournés et stockés dans des cookies *HttpOnly*.

Diagramme de séquence — Synchronisation ERP

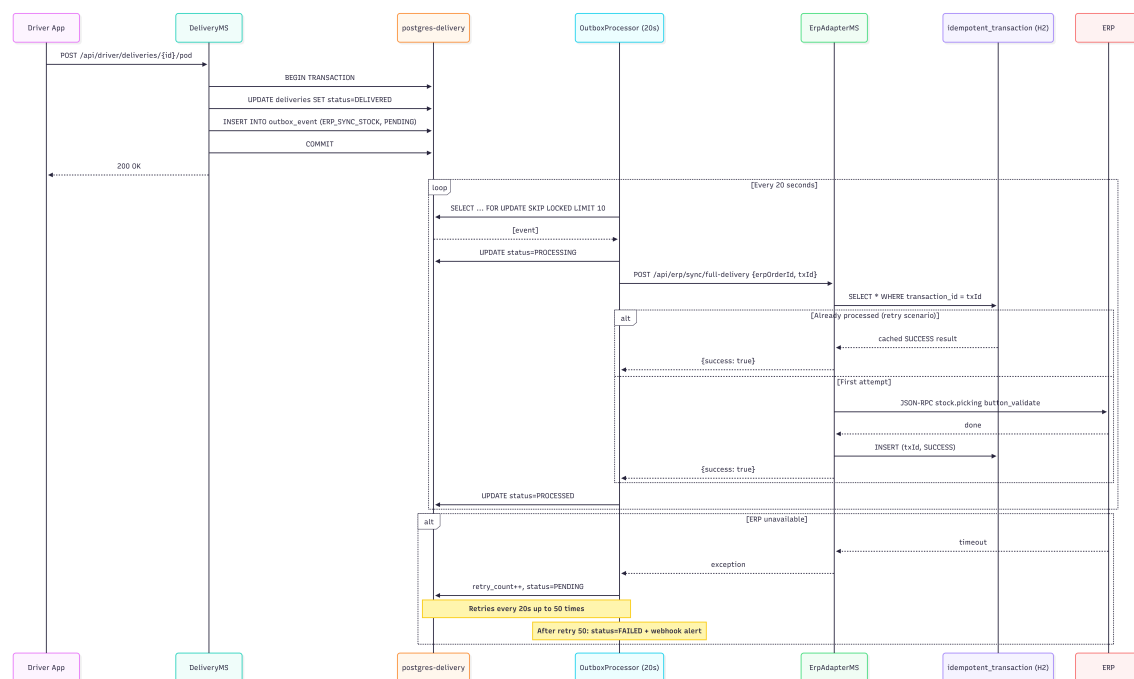


FIGURE 2.8 – Diagramme de séquence — Synchronisation ERP via patron Outbox

Lorsque le chauffeur confirme une livraison, le *DeliveryMS* écrit la mise à jour du statut et l'événement de synchronisation dans la **même transaction PostgreSQL**, garantissant l'atomicité. Un *OutboxProcessor* interroge la table toutes les 20 secondes via **FOR UPDATE SKIP LOCKED** — technique qui permet à plusieurs instances de traiter des lots différents sans collision. L'*ErpAdapterMS* reçoit l'événement, vérifie l'idempotence via une table H2 en mémoire, puis appelle le système ERP cible. En cas d'indisponibilité, le compteur `retry_count` est incrémenté et l'événement repasse en **PENDING** jusqu'à 50 tentatives, après quoi une alerte webhook est déclenchée.

3 Conclusion

Ce chapitre a permis de définir précisément le périmètre fonctionnel d'ASM Track. Nous avons identifié quatre acteurs et décrit leurs besoins fonctionnels sous forme de listes et de diagrammes de cas d'utilisation avec leurs descriptions textuelles. Les besoins non fonctionnels fixent les exigences de qualité en matière de sécurité, de performance et de fiabilité. Les diagrammes de séquence illustrent les flux d'interaction clés du système. Le chapitre suivant présente l'architecture technique retenue pour répondre à l'ensemble de ces besoins.